

Computação Paralela e Distribuída

Trabalho Académico

Solucionador do Sudoku

Versão 1.0

Segunda-feira, 15/03/2026

Conteúdo

Introdução	1
Descrição do Problema	1
Parte 1 - Implementação serial	2
Dados de Entrada	2
Dados de Saída	2
Exemplo.....	3
Parte 2 – Implementação OpenMP	3
Medição do Tempo de Execução	3
Parte 3 – Implementação MPI.....	4
O Que Entregar e Quando Entregar	4
Datas Importantes.....	4
Classificação	5

Introdução

O propósito deste trabalho académico é ganhar experiência em programação paralela em sistemas *UMA* (*Uniform Memory Access*) e multicomputadores, usando OpenMP e MPI, respectivamente. Para tal, os estudantes devem escrever uma implementação serial e duas paralelas de um solucionador do Sudoku.

Descrição do Problema

Deve escrever um algoritmo para resolver o popular quebra-cabeça Sudoku. O quebra-cabeça consiste em uma matriz parcialmente preenchida $n \times n$. Seu algoritmo precisa preencher as posições em branco com valores de 1 a n , de forma que nenhum número se repita em cada uma das n linhas, n colunas ou n quadrados de $\sqrt{n} \times \sqrt{n}$ células que dividem a matriz original.

	9	3	6	2	8	1	4	
	6						5	
	3						9	
	5						7	
	4						6	
	8						3	
	1	7	5	9	3	4	2	

2	7	1	9	5	4	6	8	3
5	9	3	6	2	8	1	4	7
4	6	8	1	3	7	2	5	9
7	3	6	4	1	5	8	9	2
1	5	9	8	6	2	3	7	4
8	4	2	3	7	9	5	6	1
9	8	5	2	4	1	7	3	6
6	1	7	5	9	3	4	2	8
3	2	4	7	8	6	9	1	5

Figura 1: Quebra-cabeça Sudoku e a solução correspondente, com $n = 9$

Nota importante: por favor, não perca tempo desenvolvendo heurísticas sofisticadas para este problema. O objectivo é implementar um algoritmo de busca simples e limpo. O que buscamos são estratégias de paralelização inteligentes. Para permitir uma avaliação justa dessas estratégias, as três implementações (serial, OpenMP e MPI) devem ser baseadas no mesmo algoritmo de busca.

Parte 1 - Implementação serial

Escreva uma implementação serial do algoritmo acima em C. Nomeie o ficheiro de origem como `sudoku-serial.c`.

Dados de Entrada

Os seus programas devem aceitar um argumento de linha de comando, que será o nome de um ficheiro contendo a matriz de instâncias do Sudoku. O formato desse ficheiro é:

- uma linha com um número inteiro, $L=\sqrt{n}$, $2 \leq L \leq 9$ (especificar L evita a raiz quadrada no código...).
- n linhas, cada uma com n números inteiros, separados por um espaço, com valores no intervalo $[0, n]$, onde 0s representam posições em branco na matriz.

Dados de Saída

O programa deve gerar (para o `stdout`) uma matriz no mesmo formato da entrada:

- n linhas, cada com n inteiros, separados por um espaço, com valores no intervalo $[1, n]$.

Os valores diferentes de zero da entrada devem ser mantidos inalterados, nenhum valor zero deve estar presente e a matriz deve estar em conformidade com as regras do Sudoku.

Caso não exista solução, o programa deve exibir a mensagem "Nenhuma solução".

Exemplo

Entrada de amostra:

```
2
0  0  3  1
3  0  0  0
4  2  0  0
1  3  0  2
```

Saída esperada:

```
2  4  3  1
3  1  2  4
4  2  1  3
1  3  4  2
```

ATENÇÃO:

- O projecto **não pode ser avaliado** a menos que siga rigorosamente estas regras de entrada e saída!

Parte 2 – Implementação OpenMP

Escreva uma implementação OpenMP deste algoritmo Sudoku, com as mesmas regras e descrições de entrada/saída. Nomeie este código-fonte como `sudoku-omp.c`.

Pode começar simplesmente adicionando directivas OpenMP, mas é livre e incentivado a modificar o código para tornar a paralelização **mais eficaz** e **mais escalável**.

ATENÇÃO: Tenha cuidado com a sincronização e o balanceamento de carga!

Nota importante: para testar a escalabilidade, executaremos este programa atribuindo valores diferentes à variável shell `OMP_NUM_THREADS`. Se substituir esse valor em seu programa, não poderemos avaliar adequadamente a escalabilidade de seu programa.

Medição do Tempo de Execução

Para garantir que todos usem a mesma medida para o tempo de execução, será usada a rotina `omp_get_wtime()` do OpenMP, que mede o tempo real (também conhecido como “wall-clock time”). Essa mesma rotina deve ser usada por todas as três versões do seu projecto. Portanto, seus programas devem ter uma estrutura semelhante a esta:

```
#include <omp.h>
<...>

int main(int argc, char *argv[])
{
    double exec_time;

    //inicialização
    exec_time =-omp_get_wtime();

    //execução do algoritmo Sudoku

    exec_time += omp_get_wtime();
    fprintf(stderr, "%.1fs\n", exec_time);

    print_result(); // usando a stream stdout!
}
```

Dessa forma, o tempo de execução contabiliza apenas o tempo de execução do algoritmo e é enviado para o erro padrão, `stderr`. O uso desses dois fluxos de saída permite que a validação dos resultados e a análise do tempo de execução sejam realizadas separadamente.

Como essa rotina de tempo faz parte do OpenMP, precisa incluir `omp.h` e compilar todos os seus programas com o `flag-fopenmp`.

Parte 3 – Implementação MPI

Escreva uma implementação MPI do algoritmo Sudoku como para o OpenMP e resolva os mesmos problemas. Nomeie este código-fonte como `sudoku-mpi.c`.

Além da sincronização e balanceamento de carga, precisará levar em consideração a **minimização do impacto dos custos de comunicação**. É encorajado a explorar diferentes abordagens para a decomposição do problema.

ATENÇÃO: Créditos extras serão dados aos grupos que apresentarem uma implementação otimizada que combine OpenMP e MPI.

O Que Entregar e Quando Entregar

O projecto será submetido em duas partes, e cada parte deverá ser entregue via Google Classroom nas datas indicadas abaixo. Para cada parte, deverá ser submetido um único ficheiro ZIP (nomeie o ficheiro como `g<n>serial.zip`, `g<n>omp.zip` e `g<n>mpi.zip`, onde `<n>` é o número do seu grupo a ser dado pelo docente). No ficheiro ZIP deve conter o código-fonte da versão serial e da versão paralela correspondente, bem como um breve relatório.

Datas Importantes

- Versão 0 (serial): 23 de Março de 2026, até as 17H.
- Versão 1 (Serial + OpenMP + Relatório): 06 de Abril de 2026, até as 17H.

- **Versão 2 (serial + MPI + Relatório):** 18 de Maio de 2026, até às 17H.

O ficheiro ZIP a ser enviado deve conter: os dois ficheiros de código-fonte, nomeados conforme indicado acima, bem como um relatório conciso e objectivo (máximo de 1 a 2 páginas) em formato PDF, nomeado report-omp.pdf (1ª fase) ou report-mpi.pdf (2ª fase). Os relatórios devem abordar os seguintes tópicos:

- Apresentar os tempos experimentais de execução das versões paralelas nos dados de entrada que serão disponibilizados (para 1, 2, 4 e 8 tarefas paralelas), os speedups, bem como outras métricas de desempenho que julgar adequadas;
- Explicar qual decomposição foi utilizada;
- Discutir quais foram as preocupações com a sincronização e porquê;
- Descrever como o balanceamento de carga foi realizado;
- Discutir os resultados de desempenho e se correspondem às suas expectativas.

Idealmente, a versão serial deve ser a mesma em ambas as submissões. No entanto, se após a primeira submissão encontrar erros ou ineficiências importantes, poderá corrigi-los e submeter uma versão serial diferente com a versão paralela MPI. Obviamente, a versão paralela MPI deve ser baseada nessa nova versão serial. Isso deve ser claramente indicado no relatório.

Nota importante: o seu projecto será avaliado na primeira aula após o prazo de entrega de cada trabalho. Portanto, é imprescindível a sua presença nessas aulas, caso contrário, seu trabalho não será avaliado. Se, por algum motivo justificável, for absolutamente impossível comparecer a essa aula, entre em contacto com o docente com antecedência e tente agendar um horário alternativo.

Classificação

Os seguintes aspectos serão levados em consideração na avaliação do projecto, não necessariamente nesta ordem:

- correcção, ou seja, se o seu programa sempre produz uma solução correcta;
- desempenho, tanto em termos de tempo (speedup), uso de memória quanto de escalabilidade;
- organização, simplicidade e clareza do código-fonte;
- originalidade.

A nota da componente prática é determinada na discussão e atribuída de acordo com o conhecimento e competências demonstradas pelo estudante sobre o trabalho submetido.

Bom trabalho!